

TP 4 — Chasse aux bugs · Binaire débranché

Semaine 4 ■ Fichiers dans NSI/TP04/ ■ 2 h

Objectifs

- Corriger des programmes bugués en lisant les messages d'erreur
- Utiliser le débogueur (exécution pas à pas)
- Manipuler le binaire « à la main » avant le cours sur les nombres

Partie A — La chasse aux bugs — bugs.py

(50 min)

Récupérer le fichier `bugs.py`. Il contient **6 programmes bugués**. Pour chacun : identifier la famille d'erreur (syntaxe / exécution / logique), corriger, et noter en commentaire ce que c'était.

```
# Bug 1 -- doit afficher les entiers de 1 à 5
for i in range(5)
    print(i)

# Bug 2 -- doit afficher le double de la saisie
n = input("Nombre ? ")
print(n * 2)

# Bug 3 -- doit afficher la moyenne des trois notes
n1 = 12
n2 = 15
n3 = 9
moyenne = n1 + n2 + n3 / 3
print(moyenne)

# Bug 4 -- doit compter de 10 à 1 puis afficher "Décollage !"
i = 10
while i > 0:
    print(i)
print("Décollage !")

# Bug 5 -- doit dire si n est dans [0, 100]
n = 150
if n >= 0 or n <= 100:
    print("dans l'intervalle")
else:
    print("hors intervalle")

# Bug 6 -- doit renvoyer le plus petit des deux nombres
def minimum(a, b):
    if a < b:
        return a
    if b < a:
        return b
print(minimum(7, 7))
```

Pour les bugs 4 et 6 : utiliser le **débogueur** ou ajouter des `print` pour observer les valeurs.

Partie B — Binaire débranché (par binômes)

(40 min)

Matériel : les 5 cartes à points (1, 2, 4, 8, 16 points — à fabriquer en papier).

1. Poser les cartes en ligne, la carte 16 à gauche. Une carte face visible = bit 1, face cachée = bit 0. Afficher : 10110. Combien de points visibles au total ?
2. Représenter : 7, 12, 21, 31. Peut-on faire 35 ? Pourquoi ?
3. **Compter en binaire** de 0 à 31 en manipulant les cartes. Que fait la carte 1 ? la carte 2 ?
4. Chaque binôme écrit un nombre ≤ 31 en binaire au tableau, l'autre le décode.
5. ★ Défi : expliquer l'astuce « pour ajouter 1 en binaire : inverser les bits de droite à gauche jusqu'au premier 0 inclus ».

Partie C — Opérateurs logiques en Python — logique.py

(30 min)

1. Vérifier en console les tables de vérité : `True and False`, `not True`, etc.
2. Écrire les fonctions `et(a, b)`, `ou(a, b)`, `non(a)` qui recalculent **and/or/not uniquement avec des if**, puis les tester sur les 4 cas.
3. Écrire `xor(a, b)` de deux façons : avec des **if**, puis en une ligne avec `!=`.
4. ★ Vérifier la loi de De Morgan par programme : une boucle qui teste `non(et(a, b)) == ou(non(a), non(b))` pour les quatre couples possibles.